

Guide pédagogique complet - Projet PFA Maintenance prédictive des GAB

Document généré le 2026-06-18 12:10 depuis le code réel du projet 'app_gab_react'.

1. Présentation générale du projet

Contexte Banque Populaire

Le projet concerne le réseau de GAB de la Banque Populaire. Un GAB est un point de contact critique entre la banque et ses clients. Lorsqu'un automate tombe en panne, l'impact n'est pas seulement technique: il touche l'expérience client, la disponibilité du cash, l'image de la banque et la charge des équipes maintenance.

Problématique métier

La problématique est: comment identifier à l'avance les GAB qui ont le plus de chances de devenir critiques, afin de planifier des interventions préventives au lieu de réagir après incident ?

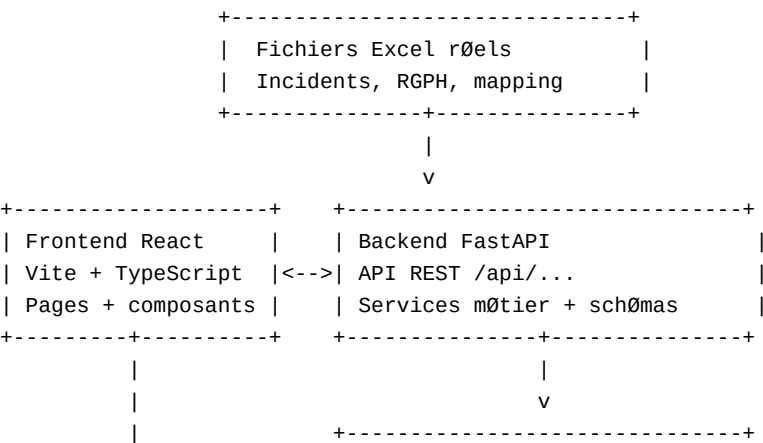
Importance de la maintenance prédictive

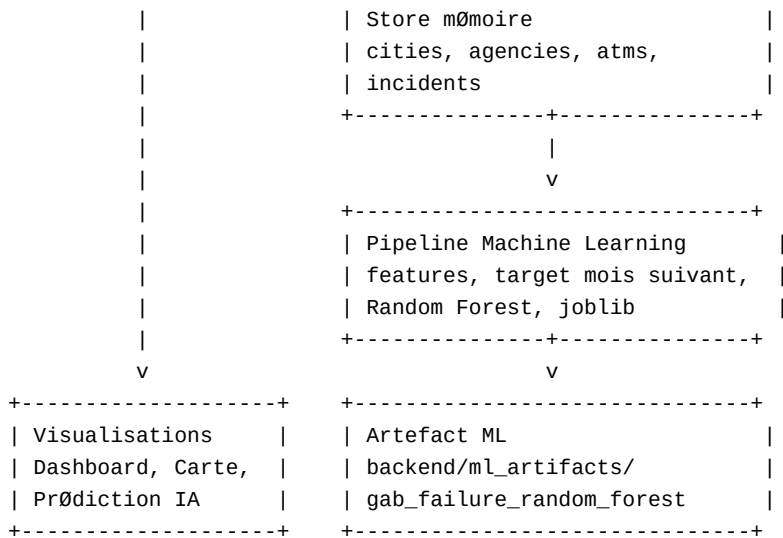
La maintenance prédictive permet de réduire les interruptions de service, prioriser les techniciens, exploiter l'historique d'incidents et transformer des fichiers Excel opérationnels en outil décisionnel.

Objectifs et résultats

Indicateur	Valeur réelle du projet
--- ---:	
Incidents chargés	9616
Agences	223
GAB	306
Villes RGPH	9
Population RGPH totale	5 593 678
MTTR	120.8 min
Motif dominant	Séparateur
Catégorie dominante	HARDWARE
GAB disponibles pour prédiction	306
GAB affichés à risque par filtre frontend	53

2. Architecture complète du système





Rôle des composants: React affiche l'interface, FastAPI expose les données et calculs, le store mémoire accède les lectures, le modèle ML estime une probabilité de panne, et les fichiers Excel sont les sources réelles.

3. Structure du projet

Frontend

Dossier/fichier	Rôle
'frontend/src/pages'	Pages: Dashboard, incidents, RGPH, carte, prédiction, recommandations.
'frontend/src/components'	Composants réutilisables: cartes KPI, charts, layout, tables, badges.
'frontend/src/services/api.ts'	Client API, timeout, gestion d'erreurs, proxy '/api'.
'frontend/src/types/api.ts'	Interfaces TypeScript des réponses backend.
'frontend/src/routes/AppRoutes.tsx'	Routes React.
'frontend/src/App.tsx'	Layout global Sidebar + Navbar + contenu.

Backend

Dossier/fichier	Rôle
'backend/app/api'	Routes FastAPI par domaine.
'backend/app/services'	Logique métier: Excel, KPIs, RGPH, IA.
'backend/app/schemas'	Schémas Pydantic.
'backend/app/store.py'	Store mémoire des villes, agences, GAB, incidents.
'backend/ml_artifacts'	Modèles sauvegardés avec joblib.
'backend/scripts'	Scripts dataset et entraînement ML.
'backend/app/main.py'	App FastAPI, CORS, routeurs, chargement startup.

4. Analyse détaillée des données

Fichiers utilisés: 'Pannes_GAB_Fusionnees(1) (1).xlsx', 'RGPH_2024_Banque_Populaire (1).xlsx', 'Mapping_Agence_Ville.xlsx', 'Statistiques_Villes.xlsx'.

Colonne incidents	Utilisation
Agence	rattachement du GAB et mapping ville
Code Agence	fallback de mapping ville
Code GAB	identifiant automate
Type GAB	variable catégorielle WINCOR/NCR

Date & heure réel	séries temporelles et fenêtres 7/30/90 jours
Durée (min)	MTTR, durée moyenne, score métier
Catégorie	dominante, encodage ML, poids métier
Motif	motif dominant, recommandations
Mois_Source	tendance mensuelle réelle

Nettoyage: '_key', '_column', '_clean_str', '_number', '_read_excel' rendent l'import robuste aux accents, alias, valeurs manquantes et formats numériques.

Données RGPH: population, hommes/femmes, tranches d'âge, 'pctOver60'. Elles servent aux KPIs démographiques, incidents par 100k habitants et features ML.

5. Feature Engineering

Feature	Définition	Formule utilisée	Intérêt métier	Impact possible
'incidents_7d'	incidents récents très courts	count cutoff-7j à cutoff	crise immédiate	augmente le risque forte hausse
'incidents_30d'	incidents du dernier mois	count cutoff-30j à cutoff	dynamique mensuelle	fortement lié à maintenance
'incidents_90d'	incidents du trimestre	count cutoff-90j à cutoff	fragilité durable	stabilise la prédiction
'monthly_frequency'	moyenne mensuelle	len(history)/observed_months	comparer les GAB	risque élevé si fréquence élevée
'atm_age'	âge approximatif	cutoff.year - year_installed	usure potentielle	peut augmenter le risque
'population'	population ville	RGPH	volume potentiel	enjeu métier plus fort
'pct_over60'	part 60+	RGPH	contexte territorial	criticité locale
'transaction_volume'	transactions/jour	estimation ou saisie	charge d'utilisation	plus de charge peut augmenter le risque
'category_code'	catégorie encodée	'_encode'	intégrer HARDWARE/AUTRES/FONDS	pattern de panne
'motif_code'	motif encodé	'_encode'	motif de panne dominant	pattern récurrent
'type_code'	type GAB encodé	'_encode'	WINCOR/NCR	différence matérielle
'city_code'	ville encodée	'_encode'	contexte géographique	effet local
'agency_code'	agence encodée	'_encode'	contexte agence	effet agence

Exemple réel GAB 1081: AL QODS, KØnitra, WINCOR, catégorie AUTRES, motif Perturbation, 4 incidents sur 7j, 19 sur 30j, 30 sur 90j, fréquence 9.0, population 1 274 663, pct 60+ 11.8, transactions estimées 3670.

6. Création de la Target

Le fichier incidents ne contient pas une colonne 'panne_future'. Le code construit donc une cible d'apprentissage:

Pour chaque mois M:

```
features = historique du GAB jusqu'à fin M
next_count = incidents du GAB au mois M+1
threshold = max(3, percentile_75 des next_count positifs)
target = 1 si next_count >= threshold, sinon 0
```

Résultats dataset ML: 2053 lignes, 561 positives, 1492 négatives, seuil critique moyen 6.0 incidents.

Cette approche simule une vraie question métier: à partir des données connues aujourd'hui, le GAB risque-t-il de devenir critique le mois prochain ?

7. Machine Learning

Random Forest est choisi car il est robuste sur données tabulaires, supporte des relations non linéaires et reste explicable en soutenance.

Features GAB

```
+--> Arbre 1: vote risque  
+--> Arbre 2: vote risque  
+--> Arbre 3: vote risque  
+--> ...  
v
```

Moyenne des votes -> probabilité de panne

Configuration réelle:

```
RandomForestClassifier(  
    n_estimators=180,  
    max_depth=8,  
    min_samples_leaf=2,  
    random_state=42,  
    class_weight='balanced'  
)
```

Sauvegarde: 'backend/ml_artifacts/gab_failure_random_forest.joblib'. L'artefact contient le modèle, les colonnes, les encodeurs, les métriques, le seuil target et la date d'entraînement.

8. Évaluation du modèle

Métrique	Valeur	Interprétation
Accuracy	0.774	77.4% des exemples bien classés
Precision	0.651	65.1% des alertes positives pertinentes
Recall	0.458	45.8% des vrais positifs détectés
F1 Score	0.537	compromis precision/recall perfectible
ROC AUC	0.803	bonne capacité à classer les GAB par risque

Lecture soutenance: le modèle est utile pour le ranking et la priorisation; le recall doit être amélioré avec plus d'historique, transactions réelles et labels maintenance.

9. Fonctionnement de la prédiction

```
Utilisateur -> React: choisit Code GAB  
React -> GET /api/predict/atms  
FastAPI -> ai_service: construit features par GAB  
FastAPI -> React: liste des GAB  
Utilisateur -> React: clique Calculer le risque  
React -> POST /api/predict  
FastAPI -> ai_service: reconstruit features depuis historique réel  
ai_service -> joblib: charge Random Forest  
Random Forest -> ai_service: probabilité  
FastAPI -> React: score, niveau, explication  
React -> Utilisateur: résultat
```

10. Analyse des APIs

Méthode	Route	Rôle
GET	'/api/data-status'	statut chargement Excel
GET	'/api/incidents/'	liste incidents

```

| GET | '/api/incidents/stats' | total incidents, agences, GAB |
| GET | '/api/incidents/top-agencies' | agences impactées |
| GET | '/api/incidents/top-atms' | GAB impactés |
| GET | '/api/incidents/monthly' | tendance mensuelle réelle |
| GET | '/api/incidents/categories' | répartition catégories |
| GET | '/api/incidents/motifs' | répartition motifs |
| GET | '/api/incidents/maintenance-kpis' | MTTR, motif, ville impactée |
| GET | '/api/rgph/stats' | KPIs RGPH |
| GET | '/api/rgph/population' | population par ville |
| GET | '/api/rgph/incidents-per-100k' | incidents normalisés |
| GET | '/api/map/' | points carte |
| POST | '/api/ai/train' | réentraîner modèle |
| POST | '/api/ai/predict' | prédiction camelCase interne |
| POST | '/api/predict' | prédiction snake_case soutenance |
| GET | '/api/predict/atms' | GAB + features formulaire |
| GET | '/api/predict/at-risk' | GAB triés par probabilité |
| GET | '/api/ai/top-critical' | top GAB critiques |
| GET | '/api/ai/recommendations' | recommandations métier |

```

Exemple 'POST /api/predict':

```

{
  "atmCode": "1081",
  "agency": "AL QODS",
  "city": "K0nitra",
  "typeGab": "WINCOR",
  "dominantCategory": "AUTRES",
  "dominantMotif": "Perturbation",
  "incidents_7d": 4,
  "incidents_30d": 19,
  "incidents_90d": 30,
  "monthly_frequency": 9,
  "population": 1274663,
  "pct_over60": 11.8,
  "transaction_volume": 3670
}

```

Réponse:

```

{
  "risk_score": 97.1,
  "risk_level": "Critique",
  "probability": 0.971,
  "explanation": "Le risque est estimé à 97.1%...",
  "recommendation": "Planifier une intervention préventive prioritaire."
}

```

11. Explication du Frontend

React organise l'interface en pages et composants. 'useEffect' charge les données, 'useState' stocke les réponses et les composants affichent cartes, graphiques et tables. 'api.ts' centralise les appels avec timeout, messages d'erreur et mapping de formats. 'types/api.ts' sécurise les contrats JSON.

La page Prédiction IA a deux usages: prédiction automatique via '/api/predict/at-risk', et prédiction manuelle via '/api/predict/atms' puis '/api/predict'.

12. Explication du Backend

FastAPI expose les routes, Pydantic valide les données, les services réalisent les calculs.

'data_loader.py' charge les Excel au démarrage et évite les rechargements inutiles.

'ai_service.py' construit les features, target, modèle, prédictions et recommandations.

13. Questions possibles du jury

Question 1. Quel problème métier résout ce projet ?

Réponse courte: Il aide à anticiper les GAB à risque de panne afin de prioriser la maintenance.

Réponse détaillée: Le projet transforme l'historique d'incidents GAB et les données RGPH en indicateurs de risque. Au lieu d'attendre qu'un automate tombe en panne, la banque peut repérer les GAB dont la fréquence d'incidents, les motifs dominants ou le contexte local augmentent la probabilité d'une panne future.

Question 2. Pourquoi avoir utilisé FastAPI ?

Réponse courte: FastAPI est rapide, typé et adapté aux APIs REST modernes.

Réponse détaillée: FastAPI permet de définir clairement les routes, les schémas Pydantic et les réponses JSON. Dans ce projet, il expose les données incidents, RGPH, carte et prédiction IA à React via des endpoints comme /api/incidents/stats et /api/predict.

Question 3. Pourquoi React côté frontend ?

Réponse courte: React facilite la construction d'une interface dynamique par composants.

Réponse détaillée: La sidebar, les cartes KPI, les graphiques, les tables et la page Prédiction IA sont des composants réutilisables. React permet de gérer les états de chargement, les erreurs API et les données renvoyées par FastAPI.

Question 4. À quoi sert TypeScript ?

Réponse courte: Il sécurise les échanges de données côté frontend.

Réponse détaillée: Les interfaces dans frontend/src/types/api.ts décrivent les réponses backend. Cela réduit les erreurs entre frontend et backend, par exemple pour PredictionAtmFeatures, AtRiskPredictionItem ou MaintenanceKpiResponse.

Question 5. Quelles données réelles sont utilisées ?

Réponse courte: Incidents GAB, RGPH, mapping agence-ville et statistiques villes.

Réponse détaillée: Les fichiers Excel dans backend/data alimentent le store mémoire: Pannes_GAB_Fusionnees, RGPH_2024_Banque_Populaire, Mapping_Agence_Ville et Statistiques_Villes. Ils donnent les incidents, agences, GAB, villes, population et coordonnées.

Question 6. Combien d'incidents sont chargés ?

Réponse courte: 9616 incidents.

Réponse détaillée: Le statut de chargement indique 9616 incidents, 223 agences, 306 GAB et 9 villes RGPH. Ces chiffres viennent de get_data_status() après chargement Excel.

Question 7. Pourquoi charger les Excel au démarrage ?

Réponse courte: Pour éviter de relire les fichiers à chaque requête.

Réponse détaillée: data_loader.py utilise un verrou et un état _loaded. load_default_data(force=False) ne recharge pas si les données sont déjà en mémoire. Cela améliore la performance des endpoints.

Question 8. Comment les colonnes Excel sont-elles détectées ?

Réponse courte: Avec une recherche robuste par alias normalisés.

Réponse détaillée: La fonction _column normalise les accents, espaces et caractères spéciaux. Cela permet de trouver Date & heure réel même si un nom proche est utilisé.

Question 9. Pourquoi utiliser un store mémoire ?

Réponse courte: Pour simplifier le prototype et accélérer les lectures.

Réponse détaillée: backend/app/store.py contient des listes cities, agences, atms et incidents. Les services lisent ces structures sans base SQL obligatoire. Pour une version industrielle, une base relationnelle ou data warehouse serait préférable.

Question 10. Qu'est-ce que incidents_7d ?

Réponse courte: Le nombre d'incidents du GAB dans les 7 derniers jours avant une date de référence.

Réponse détaillée: Dans _build_feature_record, le code compte les incidents dont reported_at est entre cutoff - 7 jours et cutoff. C'est un indicateur de tension très récente.

Question 11. Qu'est-ce que incidents_30d ?

Réponse courte: Le nombre d'incidents sur les 30 derniers jours.

Réponse détaillée: Cette variable capte la dynamique mensuelle récente d'un automate. Elle est plus stable que 7 jours mais reste proche du présent opérationnel.

Question 12. Qu'est-ce que incidents_90d ?

Réponse courte: Le nombre d'incidents sur les 90 derniers jours.

Réponse détaillée: Elle donne une vision trimestrielle, utile pour repérer un GAB durablement fragile même si la dernière semaine est calme.

Question 13. Qu'est-ce que monthly_frequency ?

Réponse courte: La fréquence moyenne mensuelle d'incidents du GAB.

Réponse détaillée: Elle est calculée par len(history) / observed_months. Elle compare équitablement des GAB ayant des historiques de durée différente.

Question 14. Pourquoi utiliser population et pct_over60 ?

Réponse courte: Pour intégrer le contexte territorial RGPH.

Réponse détaillée: Un GAB dans une zone plus peuplée ou à profil démographique sensible peut avoir un enjeu majeur plus fort. Ces variables ne remplacent pas les incidents, elles enrichissent l'analyse.

Question 15. Les transactions r  elles sont-elles disponibles ?

R  ponse courte: Non, elles sont estim  es puis modifiables.

R  ponse d  taill  e: Le code estime $\text{transaction_volume} \text{ par } 1200 + \text{monthly_frequency} * 180 + \text{population} / 1500$. Dans le formulaire, l'utilisateur peut modifier Transactions / jour.

Question 16. Comment sont encod  es les variables cat  gorielles ?

R  ponse courte: Par dictionnaires d'encodage entiers.

R  ponse d  taill  e: La fonction `_encode` convertit cat  gorie, motif, type, ville et agence en codes num  riques n  cessaires au mod  le. Les encodeurs sont sauvegard  s avec l'artefact ML.

Question 17. Comment est cr   e la target ?

R  ponse courte: $\text{target}=1$ si le GAB devient critique le mois suivant.

R  ponse d  taill  e: Pour chaque mois sauf le dernier, le code construit l'historique jusqu'   la fin du mois, puis compte les incidents du mois suivant. Si ce count d  passe un seuil mensuel, target vaut 1.

Question 18. Quel est le seuil critique ?

R  ponse courte: Un seuil bas   sur les donn  es: $\max(3, \text{percentile } 75 \text{ des counts positifs du mois suivant})$.

R  ponse d  taill  e: Cette r  gle   vite un seuil arbitraire fixe trop faible ou trop haut. Le seuil critique moyen calcul   dans ce projet est 6.0 incidents.

Question 19. Pourquoi pr  dire le mois suivant ?

R  ponse courte: Parce que c'est proche d'une d  cision de maintenance pr  ventive.

R  ponse d  taill  e: La banque doit planifier des interventions avant que le risque ne se mat  rialise. La logique mois suivant simule une question op  rationnelle: quels GAB risquent de devenir probl  matiques le mois prochain ?

Question 20. Pourquoi Random Forest ?

R  ponse courte: Il est robuste, explicable et fonctionne bien avec variables mixtes.

R  ponse d  taill  e: Random Forest combine plusieurs arbres de d  cision. Il supporte des relations non lin  aires et r  duit le surapprentissage par agr  gation de nombreux arbres.

Question 21. Qu'est-ce qu'un arbre de d  cision ?

R  ponse courte: Une suite de questions oui/non sur les variables.

R  ponse d  taill  e: Un arbre peut par exemple s  parer les GAB avec $\text{incidents_90d} \geq \text{lev  }$, puis $\text{monthly_frequency} \geq \text{lev  }$, puis cat  gorie HARDWARE. Chaque feuille donne une probabilit   de risque.

Question 22. Qu'est-ce qu'une for  t al  atoire ?

R  ponse courte: Un ensemble d'arbres entra  n  s sur des sous-  chantillons.

R  ponse d  taill  e: Chaque arbre vote. La probabilit   finale correspond    la proportion d'arbres qui classent le GAB comme critique futur. Cela stabilise la pr  diction.

Question 23. Où est sauvegardé le modèle ?

Réponse courte: backend/ml_artifacts/gab_failure_random_forest.joblib.

Réponse détaillée: Le service utilise joblib.dump pour sauvegarder l'artefact complet: modèle, colonnes de features, encodeurs, métriques, seuil target et date d'entraînement.

Question 24. Que contient l'artefact ML ?

Réponse courte: Modèle, features, encodeurs, métriques et métadonnées.

Réponse détaillée: Dans ai_service.py, artifact contient kind, model, featureColumns, encoders, metrics, xgboostMetrics, targetThreshold et trainedAt.

Question 25. Quelle est l'accuracy obtenue ?

Réponse courte: 0.774.

Réponse détaillée: Cela signifie que 77.4% des exemples de test sont classés correctement. Mais cette métrique seule peut être trompeuse si les classes sont déséquilibrées.

Question 26. Quelle est la precision ?

Réponse courte: 0.651.

Réponse détaillée: Quand le modèle annonce un GAB à risque, il a raison environ 65.1% du temps sur l'échantillon de test. C'est utile pour éviter trop de fausses alertes.

Question 27. Quel est le recall ?

Réponse courte: 0.458.

Réponse détaillée: Le modèle détecte environ 45.8% des vrais cas positifs. C'est améliorable, mais cohérent avec un historique court et une target reconstruite.

Question 28. Quel est le F1 score ?

Réponse courte: 0.537.

Réponse détaillée: Le F1 équilibre precision et recall. Il montre que le modèle est utile mais pas parfait; il doit être considéré comme une aide à la décision.

Question 29. Quel est le ROC-AUC ?

Réponse courte: 0.803.

Réponse détaillée: Un ROC-AUC de 0.803 indique une bonne capacité de ranking: le modèle ordonne globalement mieux les GAB à risque que le hasard.

Question 30. Pourquoi ne pas prétendre que le modèle est industriel ?

Réponse courte: Parce que certaines données métier manquent.

Réponse détaillée: Il manque les transactions réelles par GAB, les logs techniques détaillés, les interventions maintenance et une vraie étiquette panne future validée.

Question 31. Comment fonctionne POST /api/predict ?

Réponse courte: Il reçoit les features d'un GAB et renvoie score, niveau, probabilité et explication.

Réponse détaillée: Si atmCode est fourni, le backend reconstruit les features depuis l'historique réel du GAB, puis applique le modèle sauvegardé.

Question 32. Pourquoi le frontend passe par /api ?

Réponse courte: Pour utiliser le proxy Vite et éviter les problèmes CORS/Safari.

Réponse détaillée: api.ts définit `BASE_URL = VITE_API_URL ?? '/api'`. En développement, Vite proxifie /api vers FastAPI.

Question 33. Que fait GET /api/predict/atms ?

Réponse courte: Il retourne tous les GAB disponibles avec leurs features calculées.

Réponse détaillée: La page Prédiction IA utilise cet endpoint pour remplir automatiquement le select Code GAB, l'agence, la ville, le type, les incidents 7/30/90 jours et le contexte RGP.

Question 34. Que fait GET /api/predict/at-risk ?

Réponse courte: Il retourne les GAB triés par probabilité décroissante.

Réponse détaillée: Le frontend filtre ensuite les GAB Critique, Élevé ou probabilité > 70% pour la section automatique.

Question 35. Pourquoi garder le formulaire manuel ?

Réponse courte: Pour tester un GAB précis avec transactions ajustées.

Réponse détaillée: La banque peut sélectionner un GAB réel, conserver les features historiques et modifier uniquement Transactions / jour si elle dispose d'une meilleure estimation métier.

Question 36. Comment sont calculés les niveaux de risque ?

Réponse courte: Faible <40, Moyen 40-59, Élevé 60-79, Critique >=80.

Réponse détaillée: La fonction `_risk_category` applique ces seuils. `POST /api/predict` utilise `probability*100` comme `risk_score`.

Question 37. Quelle différence entre riskScore et failureProbability ?

Réponse courte: La probabilité vient du modèle; le score est l'expression métier sur 100.

Réponse détaillée: Dans la prédiction manuelle, `riskScore = probability*100`. Dans la liste automatique, `riskScore` combine aussi un score métier heuristique avec la probabilité, les incidents et la durée.

Question 38. Pourquoi la liste automatique peut avoir Élevé avec probabilité très forte ?

Réponse courte: Le niveau dépend du score métier combiné, pas seulement de la probabilité.

Réponse détaillée: Dans `_latest_rows_for_atms`, le `riskScore` automatique combine `incidents_90d`, durée moyenne, diversité de catégories, poids de catégorie et probabilité ML.

Question 39. Où sont les routes frontend ?

Réponse courte: `frontend/src/routes/AppRoutes.tsx`.

Réponse détaillée: Ce fichier mappe les chemins `/`, `/incidents`, `/demographics`, `/map`, `/predictions` et `/recommendations` vers les pages correspondantes.

Question 40. Quel composant gère la navigation ?

Réponse courte: Sidebar.tsx et Navbar.tsx.

Réponse détaillée: Sidebar affiche les sections métier; Navbar contient le statut IA, le bouton menu et les notifications.

Question 41. Comment les erreurs API sont affichées ?

Réponse courte: api.ts construit des messages détaillés.

Réponse détaillée: fetchJson ajoute l'URL, le status HTTP, le détail backend, les erreurs timeout et les erreurs réseau/CORS.

Question 42. Pourquoi les loaders ne restent plus infinis ?

Réponse courte: Les appels utilisent catch/finally côté pages.

Réponse détaillée: Chaque page met fin au loading même en cas d'erreur et affiche un message compréhensible au lieu d'un spinner permanent.

Question 43. Comment fonctionne le Dashboard ?

Réponse courte: Il agrège stats incidents, RGPH, top agences, catégories, KPIs maintenance et données IA.

Réponse détaillée: Dashboard.tsx appelle plusieurs endpoints via Promise.all, puis alimente les KpiCard et les graphiques.

Question 44. Comment fonctionne la carte interactive ?

Réponse courte: Elle agrège les incidents par ville et colore selon le risque.

Réponse détaillée: backend/app/services/data_service.py enrichit les points avec gabCount, averageRiskScore et riskCategory; React/Leaflet affiche les points et popups.

Question 45. Qu'est-ce que MTTR ?

Réponse courte: Mean Time To Repair, durée moyenne de résolution.

Réponse détaillée: Dans ce projet, il est basé sur Durée (min). La valeur actuelle est 120.8 minutes.

Question 46. Pourquoi ne pas calculer MTBF ?

Réponse courte: Les données ne donnent pas un temps de fonctionnement fiable entre pannes.

Réponse détaillée: Sans logs de disponibilité ou dates de remise en service fiables, MTBF ou disponibilité réseau seraient inventés. Le projet se limite aux KPIs calculables.

Question 47. Pourquoi utiliser Mois_Source ?

Réponse courte: Pour respecter les mois réellement présents dans les fichiers.

Réponse détaillée: Le graphique mensuel utilise les mois présents, de Sep 2025 à Avr 2026, sans ajouter des mois artificiels à zero.

Question 48. Comment répondre si le jury critique les métriques ?

Réponse courte: Dire que le modèle est une première version explicable et améliorable.

Réponse détaillée: Les métriques sont honnêtement présentées. Le ROC-AUC est bon, le recall reste perfectible. L'objectif PFA est de démontrer un pipeline complet et une aide à la décision, pas un modèle industriel final.

Question 49. Qu'est-ce qui rend le projet Big Data & IA ?

Réponse courte: L'intégration multi-sources, la préparation de features temporelles et le modèle prédictif.

Réponse détaillée: Le projet combine Excel incidents, RGPH, mapping, agrégations métier, API REST, visualisations et Random Forest sauvegardé.

Question 50. Comment améliorer le modèle ?

Réponse courte: Ajouter transactions réelles, logs techniques, maintenance et plus d'historique.

Réponse détaillée: Avec plus de données labellisées, on pourrait comparer XGBoost, tuning d'hyperparamètres, validation temporelle stricte et monitoring en production.

Question 51. Quelle est la réponse la plus importante à mémoriser ?

Réponse courte: Le projet transforme des incidents historiques en actions de maintenance prioritaires.

Réponse détaillée: Tout part de la donnée réelle: incidents et RGPH. Le backend nettoie et calcule les features; le modèle estime le risque; le frontend rend les résultats exploitables par la banque.

14. Forces du projet

Points forts techniques: architecture séparée React/FastAPI, API typée, données réelles, import Excel robuste, Random Forest sauvegardé, prédiction manuelle et automatique, visualisations métier, gestion d'erreurs claire, proxy Vite '/api'.

Points forts métier: priorisation des GAB à risque, motifs et catégories dominants, KPIs maintenance, contexte RGPH, recommandations exploitables.

15. Limites du projet

Historique limité à Sep 2025 - Avr 2026; transactions réelles absentes; logs techniques absents; historique maintenance incomplet; target reconstruite; store mémoire adapté au prototype; recall améliorable.

16. Améliorations futures

Ajouter transactions réelles, logs techniques, interventions maintenance, XGBoost, tuning d'hyperparamètres, validation temporelle stricte, monitoring de durée, PostgreSQL/data warehouse, authentification, entraînement automatique et données temps réel.

17. Résumé final à mémoriser

Ce projet transforme l'historique réel des incidents GAB et les données RGPH en outil d'aide à la décision. Le backend nettoie les Excel, calcule des features temporelles, entraîne un Random Forest et expose les prédictions via API. Le frontend React affiche les KPIs, les analyses, la carte, les recommandations et une page de prédiction IA avec deux usages: identifier automatiquement les GAB à risque et tester manuellement un GAB précis.

Phrase de soutenance possible:

> Mon projet transforme l'historique réel des incidents GAB et les données RGPH en outil d'aide à la décision. L'objectif n'est pas seulement d'afficher des statistiques, mais d'anticiper les automates les plus susceptibles de devenir critiques afin de prioriser la maintenance préventive.